

Gazebo Camera Bottleneck Evidence

Why the current laptop is useful development hardware, but not enough for full ROS 2 + Gazebo camera-sensor campaigns.

Prepared for thesis compute discussion.

Decision Ask

I need access to a workstation for ROS 2 / Gazebo camera-sensor experiments.

This is not a request for a nicer laptop. The evidence points to a specific bottleneck:

- Synthetic camera rendering in Gazebo.
- Sensor topic publication and ROS/Gazebo bridging.
- CPU contention with YOLO, planning, logging, and simulator processes.
- A rendering path that must be verified, not assumed from the presence of an NVIDIA GPU.

Runtime Stack

Gazebo warehouse world

- > external RGB camera, 1280x720, target 5 Hz
- > ros_gz_bridge
- > YOLO detector
- > pixel-to-BEV state update
- > EFE planner
- > command + logging

In a real robot, the physical camera produces frames directly. In Gazebo, every frame must be rendered, copied, serialized, bridged, and then processed.

Current Laptop Context

The laptop is reasonable for development, but it is not a simulation workstation.

- CPU: Intel Core i7-9750H, 6 cores / 12 threads, 45 W mobile processor.
- GPU: NVIDIA Quadro P2000 Mobile plus Intel UHD Graphics 630.
- RAM: 16 GB class system.
- Era/class: 2019 mobile workstation hardware.

Interpretation: good enough to develop the method, weak for full synthetic-camera simulation plus perception and planning at target rate.

Source: Intel i7-9750H product specifications.

Cleanup Already Done

Before asking for compute, I removed avoidable runtime overhead:

- RGB camera capped at 5 Hz.
- Unnecessary segmentation camera disabled.
- ROS `/clock` throttled to 50 Hz.
- Raw Gazebo clock moved to `/clock_full`.
- Physics kept unchanged:
- `max_step_size = 0.001`
- `real_time_update_rate = 1000`
- Camera resolution kept at 1280x720 to avoid recalibration and preserve thesis comparability.

Local Measurement

Single timing probe after clock cleanup, 2026-06-19.

Condition | Observation

Isolated Gazebo | RGB camera reaches intended `5.00 sim-Hz`

Before clock throttle | `/clock` around `1930 Hz`; camera under full load around `1.7-2.5 sim-Hz`

After clock throttle | `/clock\_full` around `576 Hz`; ROS `/clock` exactly `50.0 Hz`

Full stack after throttle | camera/perception median `1.67 sim-Hz`

Detector callback while driving | median `125 ms`

Frame age while driving | median `0.119 s`

Planner pixel correction age | median `0.917 s`

Run path: `logs/visibility\_comparison/\_clockthrottle\_codex/.../experiment\_20260619\_133958`

What The Measurement Shows

The clock cleanup worked, but it did not unlock the camera.

- ROS `/clock` callback churn was real and was fixed.
- The full stack still cannot sustain the configured 5 Hz camera/correction rate.
- YOLO latency alone does not explain the camera period: median detector callback is about 125 ms, while camera updates arrive around every 600 ms.
- The remaining bottleneck is consistent with Gazebo camera rendering, sensor publication, bridge overhead, and CPU contention.

The result is stronger after cleanup because the easy system-level overhead has already been removed.

External Evidence 1: Strong GPU, Still Camera-Limited

Gazebo Sensors issue #332 reports a user with:

- RTX 3080 GPU.
- Ryzen 5800X CPU.
- Ubuntu 22.04, ROS 2 Humble, Gazebo Garden.
- Three 1920x1080 cameras configured for 30 FPS.

Observed result:

- Three cameras publish at about 15 Hz each.
- Even one camera fluctuates around 23 Hz instead of 30 Hz.
- Running headless and removing scene content did not fix it.

Source: <https://github.com/gazebosim/gz-sensors/issues/332>

External Evidence 1: Maintainer Explanation

In the same Gazebo Sensors issue, a maintainer explains that GUI rendering and server-side sensor rendering are different paths.

Key point:

- GUI rendering has its own process/thread.
- Server-side rendering sensors update through the sensor rendering path.
- Bottlenecks can happen during GPU-to-CPU readback, serialization, and message publication.

This matches our symptom: the simulator can display/run, but camera sensor publication under load does not meet the configured rate.

Source: <https://github.com/gazebosim/gz-sensors/issues/332>

External Evidence 2: RTX 3060, 20 FPS Becomes 5-6 FPS

Gazebo Sim issue #2796 reports a user with:

- 12-thread CPU.
- NVIDIA RTX 3060.
- Ogre2 rendering.
- Real hardware.

Observed result:

- After subscribing to two camera topics and an IMU, configured 20 FPS cameras effectively became 5-6 FPS.
- CPU and GPU did not show full utilization.
- The same behavior was seen across Fortress, Garden, and Harmonic, with Fortress somewhat faster.

This is close to our pattern: configured camera rate is not achieved once the full consumer stack is active.

Source: <https://github.com/gazebosim/gz-sim/issues/2796>

External Evidence 3: Bridge And Sensor Publication Cost

ros_gz issue #368 reports a Create 3 simulation using GPU lidar sensors bridged to ROS 2.

Observed result:

- Desired update rate: 62 Hz.
- ROS observed rates: about 35 Hz and 47 Hz.
- Gazebo RTF dropped to about 40-60 percent.
- Disabling the sensor bridges gave a steady RTF around 90 percent.

This supports the idea that the Gazebo sensor publication plus ROS bridge path can reduce simulation throughput, not just the perception algorithm.

Source: https://github.com/gazebo-sim/ros_gz/issues/368

External Evidence 4: GPU Present Does Not Mean Gazebo Uses It

Gazebo Sim issue #2595 reports an NVIDIA RTX 3060 system where Gazebo fell back to CPU/llvmpipe rendering.

Relevant symptoms:

- `glxinfo` showed Mesa `llvmpipe`.
- Gazebo did not appear as a GPU process.
- Framerates were low.

Maintainer guidance focused on checking the actual OpenGL renderer and comparing forced software rendering against normal rendering.

Conclusion: a compute request must require verified Gazebo OpenGL/EGL GPU rendering, not just CUDA availability.

Source: <https://github.com/gazebosim/gz-sim/issues/2595>

Why A Real System Can Be Lighter

Real robot:

physical camera -> image driver -> YOLO -> estimator -> planner

Gazebo camera simulation:

physics -> synthetic scene render -> GPU/CPU readback

-> Gazebo transport -> ROS bridge -> image message

-> YOLO -> estimator -> planner

Gazebo adds a synthetic image-generation workload that a real robot camera does not have. The real camera is a sensor; the simulated camera is a renderer plus a sensor pipeline.

Compute Request

Minimum sensible workstation:

- Native Ubuntu/Linux.
- 16+ modern CPU cores.
- 64 GB RAM.
- NVIDIA RTX 4070 / 4080, RTX A4000 / A5000, or better.
- NVMe storage.
- Verified OpenGL/EGL GPU rendering for Gazebo camera sensors.

Important: CUDA access alone is not sufficient. Gazebo camera rendering must use the GPU render path.

Acceptance Test For New Compute

Before relying on new hardware, run the same A/B checks:

Check | Target

``glxinfo -B`` | OpenGL renderer is NVIDIA/RTX, not llvmpipe

Isolated Gazebo | RGB camera reaches ``5.00 sim-Hz``

Full stack | camera/correction rate close to 5 Hz

ROS clock | ``/clock`` stays throttled at 50 Hz

CPU | Gazebo, YOLO, planner, logger have headroom

Logs | same campaign config, same 1280x720 camera, same physics

If the new machine still fails, that is publishable evidence of a simulator-stack limitation rather than a laptop limitation.

If Compute Is Not Available

Continue method development on the laptop, but frame the runtime result correctly:

- The laptop validates software integration and method behavior.
- It does not provide enough headroom for full synthetic-camera campaign runs.
- Do not lower camera resolution unless forced; it complicates calibration and thesis comparability.
- If needed, report a simulator-induced timing limitation and support it with the A/B evidence.

This keeps the thesis honest without spending more time on low-yield Gazebo tuning.

Bottom Line

The current result is not "my code is too slow" or "the laptop is simply bad."

The better conclusion is:

The method runs, but the current laptop cannot run Gazebo synthetic camera rendering, ROS/Gazebo bridging, YOLO, planning, and logging with enough headroom to sustain the target 5 Hz correction loop.

The compute need is specific and testable: verified GPU-accelerated Gazebo sensor rendering plus modern CPU headroom.

Sources

- Local timing run: ``logs/visibility_comparison/_clockthrottle_codex/route_apron_to_a3_mid/C2/seed0/experiment_20260619_133958``
- Intel i7-9750H specs:
<https://www.intel.com/content/www/us/en/products/sku/191045/intel-core-i79750h-processor-12m-cache-up-to-4-50-ghz/specifications.html>
- Gazebo Sensors #332, slow camera update rate: <https://github.com/gazebosim/gz-sensors/issues/332>
- Gazebo Sim #2796, camera subscriptions reduce effective FPS: <https://github.com/gazebosim/gz-sim/issues/2796>
- **ros_gz #368, bridge performance issues:** https://github.com/gazebosim/ros_gz/issues/368
- Gazebo Sim #2595, GPU fallback to CPU/llvmpipe: <https://github.com/gazebosim/gz-sim/issues/2595>
- Gazebo architecture docs: <https://gazebosim.org/docs/fortress/architecture/>